

Assessor's Feedback (may be delivered in line with the submission)

Were the learning outcomes met?

Yes ☐ If not, what was not met:

Assessor's response to the student's submission, request for feedback and / or self-assessment (feedback):

What specific actions should the student undertake to progress their learning? (feedforward):

Please take this and other feedback to your next academic tutorial to plan your future work.

Database Management System for an Online Grocery Delivery Service

Module Code: LDS7002M - Database Systems & Security

Level of Study: 7

Submission Date: January 20, 2026

Word Count: 6776 words

Table of Contents

1. Executive Overview.....	7
2. Project Introduction.....	7
2.1 Context and Rationale.....	7
2.2 Aims and Intended Outcomes.....	8
3. Business Requirements and Analysis.....	8
3.1 Core Functional Requirements.....	8
3.2 Supporting Non-Functional Requirements.....	9
4. Database Design Strategy.....	10
4.1 Modelling approach and Entity Discovery.....	10
4.2 Defining Entity Relationships.....	10
4.3 Relationship Definition.....	11
5. Conceptual Database Model.....	13
6. Normalization Stages.....	14
6.1 Purpose and Principles of Normalisation.....	14
6.2 First Normal Form (1NF).....	15
6.3 Second Normal Form (2NF).....	15
6.4 Third Normal Form (3NF).....	16
7. Physical Database Implementation.....	18
7.1 Platform choice and table construction.....	18
7.2 Data Population and Indexing.....	20
8. SQL Queries and Analytical findings.....	21
8.1 Query 1: Total Sales by Product Category.....	21
8.2 Query 2: Top 10 Most Popular Products.....	22
8.3 Query 3: Customer Order History.....	23
8.4 Query 4: Current Inventory Levels.....	25
8.5 Query 5: Revenue by City.....	26
9. Data Security and User Access.....	28
9.1 Access Control and User Role Permissions.....	28
9.2 Security Safeguards and Integrity Measures.....	28
9.3 Protecting Sensitive Information.....	29
9.4 Data Integrity Controls.....	29
9.5 Audit and Accountability.....	29
10. Testing and Verification Procedures.....	29
11. Evaluation and Forward planning.....	30

11.1 Opportunities and Further Developments.....	30
11.2 Final Summary and Reflection.....	31
12. Conclusion.....	33
13. References.....	34
14.SQL Implementation Screenshots.....	35

1. Executive Overview

A detailed look at how a relational database was built begins here, shaped by the needs of a grocery delivery platform active in several urban areas. From one city to another, product ranges shift - this complexity guided early architecture choices. Customer order patterns influenced structural decisions behind the scenes. Efficiency emerged as a priority when handling live updates to stock levels each day. Security measures were woven into workflows so personal details stay protected. Data accuracy became central during testing phases. What started as sketches turned into functional components through iterative refinement.

Starting with business needs, the design took shape through careful analysis of operational demands. Following this, an Entity–Relationship framework emerged - mapping core elements like Products, Categories, Customers, Orders, Transactions, Inventory, and Cities. Instead of jumping straight into coding, modellers used relational principles guided by a stepwise approach. This structure then advanced to Third Normal Form, reducing repetition while strengthening reliability across records.

A structure for storing data came into being through SQL, shaping tables with clear links between them - primary keys tied to foreign ones - to support reliable connections across records. Test information filled these tables, making sure operations could be checked under realistic conditions. From this foundation emerged a series of targeted questions posed to the data, pulling out patterns like total income per product group, what customers bought over time, current stock levels, top-selling items, and earnings broken down by urban area.

From the start, safeguards for personal data and who can access it shaped how the system was built. Looking ahead, room remains to grow - especially by making alert functions smarter and expanding capacity over time.

2. Project Introduction

2.1 Context and Rationale

The sharp rise in online grocery shopping—accelerated by the pandemic—has changed how customers buy food, increasing demand for fast, secure digital services. To manage large volumes of products, customers, orders and payments, online retailers rely on accurate and well-organised data systems.

This project develops a relational database for a multi-location grocery delivery company. It supports day-to-day tasks like ordering, payments and stock control while protecting customer information and meeting GDPR requirements. The database was designed using standard methods: gathering requirements, creating ER diagrams, normalising to 3NF and implementing the final structure in SQL, ensuring reliability, security and room for future growth.

2.2 Aims and Intended Outcomes

This project develops a reliable relational database for an online grocery delivery service, designed to support daily operations and protect customer data. The work follows a practical process: analysing business activities such as procurement, ordering and transactions, identifying data needs and mapping workflow information.

From this, core entities and their relationships are modelled into an ER diagram, which forms the basis of a structured database. The design is refined to Third Normal Form to remove duplication and improve data integrity. The system is then implemented in SQL with tables, primary and foreign keys, constraints and realistic sample data.

Queries provide insight into sales, customer activity and stock changes over time. Security safeguards restrict access to sensitive information, though performance and protection can be improved in future through indexing, monitoring and stronger encryption.

3. Business Requirements and Analysis

3.1 Core Functional Requirements

The database must support the full operational flow of an online grocery delivery service, with all functions guided by real business processes. It needs to manage customer activity from browsing to purchase, track orders accurately, maintain up-to-date product and stock information, and process payments without errors. The system must work reliably at volume, maintain data accuracy, and protect sensitive information throughout daily operations.

Managing Customers & Orders

- Store customer contact details: **name, address, email, phone**
- Display products clearly for browsing and selection
- Allow smooth cart management and checkout
- Record orders with **customer, time, location, items, quantities**
- Maintain customer–order history to track buying patterns

Product & Inventory Management

- Store product attributes: **name, category, description, price, stock**
- Update stock automatically after sales
- Assign each product to **one category**
- Add new products and new categories easily
- Continuously monitor inventory levels

Handling Payments & Purchases

- Record sales with **units sold, price, and totals**
- Auto-update inventory and revenue figures
- Correctly link **customer, order, and transaction**
- Support billing and financial reporting accuracy

Reporting & Business Intelligence

- Revenue by category/time period
- Top-selling products
- Customer purchase frequency
- Real-time stock levels
- Income by city and date range
- Enable sorting, filtering and grouping in queries

Security & Access

- Restrict access by user role
- Protect customer/order data from unauthorized updates
- Ensure confidentiality and integrity of stored information

3.2 Supporting Non-Functional Requirements

How well a database works often depends on invisible rules guiding its design. Performance, safety, and consistency come into play only when these background conditions are clearly set. Smooth operation in daily business tasks links directly to how strictly those limits are followed. Reliability emerges not just from code, but from defined boundaries shaping behavior.

Performance

- Database must process customer, product and order activity quickly
- Reports (sales, stock, etc.) should run fast enough for real-time decisions

Scalability

- Should handle growth in products, customers, and order volume
- Must support expansion into new locations with minimal structural change

Availability & Reliability

- System must run consistently with minimal downtime
- Core tasks (orders, inventory, transactions) should always work correctly

Data Integrity

- Use primary/foreign keys and constraints to prevent invalid data

- Maintain 3NF to reduce errors and ensure consistency across tables

Security & Privacy

- Protect personal data in line with GDPR
- Access to sensitive information must rely on role-based permissions

Maintainability

- Structure should be clear so updates and additions are easy
- Use documentation/comments to support troubleshooting and teamwork

Usability

- Tables and fields must be well named and logically structured
- Queries and reports should be clear and easy for business users to interpret

4. Database Design Strategy

4.1 Modelling Approach and Entity Discovery

Ahead of any coding, real-world tasks drove how data would later connect inside the system. What mattered most were daily activities - customers searching items, purchases moving through checkout, stock levels shifting, reports forming from transactions. Instead of jumping into software specifics, early sketches mapped people, things, actions using boxes and lines on paper. These drawings followed a classic method where each circle or rectangle stood for something actual but stayed separate from tech choices down the road.

From that early sketch, each piece turned into its own table, relationships held tight by key links. Through stages of tightening structure - up to 3NF - repeats faded, accuracy grew. Step after step, the blueprint took shape, ready for real SQL build and deeper analysis down the line.

4.2 Defining Entity Relationships

From the business area, eight main players came into view

A location in a urban area stands for where things get delivered. It helps figure out how much money comes from different parts of the region.

Customer – Stores customer identity and contact information.

A single category pulls together items that go together. It makes finding things easier when looking around. Think of it like a label that helps track what sells and where. Sorting this way also simplifies how results are shared later.

Built around what the item is called, how much it costs, and where it fits in grouping - this part holds details tied directly to a single product.

What you've got, where it sits - keeps count of items in storage by product. A clear view of what's on hand shows up right away.

A single purchase gets recorded here, along with its overall information. The system holds everything tied to that transaction in one place.

A single thing someone buys shows up here, along with how many they want. Each purchase gets broken down into these pieces when stored.

A single event captures money movement, updates account condition. Payment happens here, balance shifts accordingly. This entry marks when funds change hands, records outcome clearly. Movement of cash gets noted at this point, state adjusts after.

From stock checks to checkout data, one piece leads to the next. Each step ties into what follows, shaping how things move and get counted. Order details flow through systems until numbers turn into reports. What starts with shelves ends in summaries that show what worked.

4.3 Relationship Definition

Starting off, the Entity-Relationship model outlines connections between data elements in the online grocery delivery setup. What ties together customer actions is their ability to initiate orders through structured links. Moving on, product availability depends on inventory records that update once a sale occurs. Payment handling runs parallel to order confirmation, ensuring transactions align with purchases. Behind each purchase lies a chain of updates - stock counts shift the moment an item sells. These linkages ensure information flows without overlap or gap

Customer – Order (1:M)

A single customer might submit several orders over time. Yet each order ties back to just one individual who placed it.

(Customers place orders)

City – Customer (1:M)

Some cities include numerous signed-up clients. Each client is linked to just a single location.

People who reside in urban areas form the customer base

City – Order (1:M)

A connection between orders and the destination city allows reports by region. Delivery location defines where each order belongs. Reporting works better when tied to specific urban areas. Each transaction aligns with its drop-off point. Geographic clarity supports accurate summaries across districts.

City locations receive order placements

Category – Product (1:M)

One category holds several items, yet every item fits into just a single group.

Grouping happens according to product type

Product – Inventory (1:M)

A single item could appear in several stock logs, while each log belongs strictly to that one item.

Records multiply per product, yet split no further than their source.

Stock quantities exist for every product

A single order may include multiple items. Each item links back to one specific order

One order contains several individual lines. Every item ties exclusively to just one purchase record.

(Orders contain many product lines)

Product – OrderItem (1:M)

A single item can show up across several entries in orders, yet every entry points to just that one item.

Items ordered contain the products listed

Order – Transaction (1:M)

A single order can lead to multiple payments, though every transaction ties back to just one purchase.

What matters is that payments never split across orders.

Payment handling occurs once orders are placed

From product availability to financial summaries, each step connects smoothly across inventory, transactions, and delivery tracking. Reporting follows naturally by customer and location once orders are processed and settled. Movement through the system relies on linked stages that start with stock and end with insights.

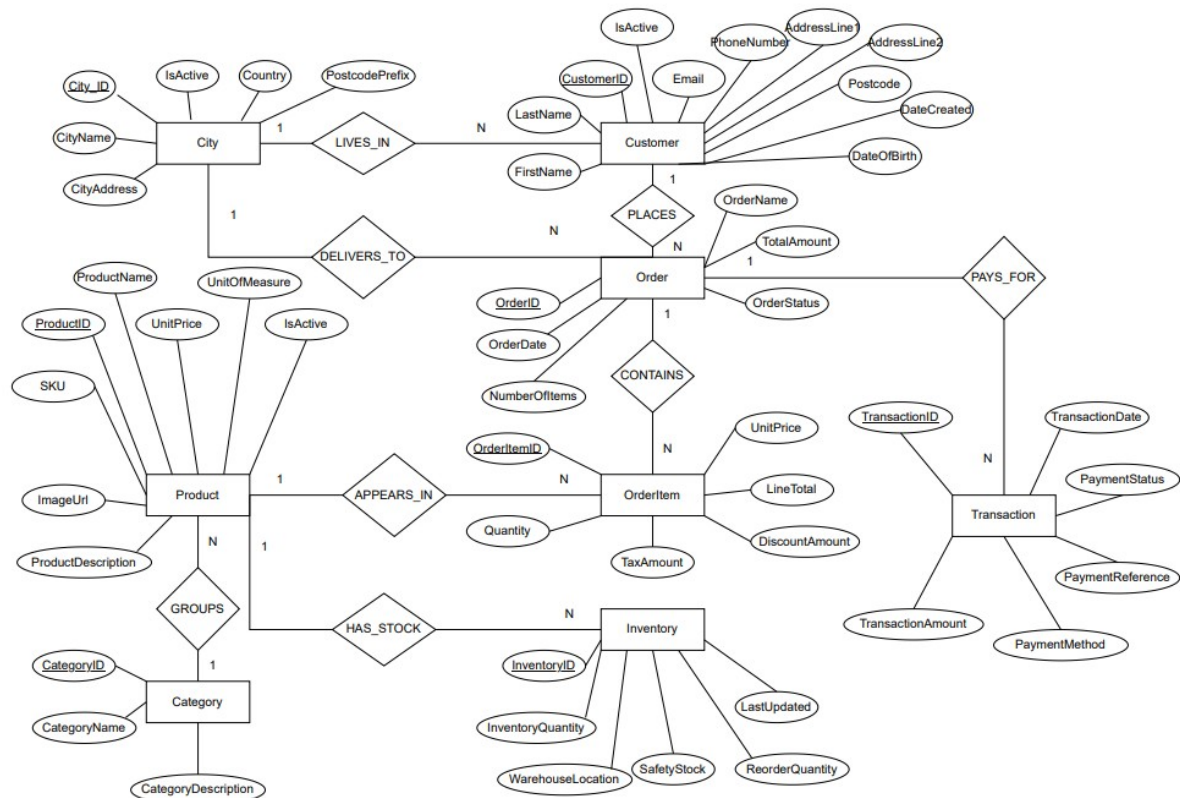


Figure 1: Entity-Relationship Diagram showing eight interconnected entities

5. Conceptual Database Model (ERD)

Starting off, the Entity–Relationship (ER) model outlines what data the online grocery delivery system needs in broad terms. In Chen notation, key business components appear as entities, each carrying specific descriptive features. Connections between these parts show how they interact within the structure.

Apart from City, there is Customer - each tied by identifiers like CustomerID or OrderID - to hold vital details about people, goods, locations, purchases, and stocks. Attributes store practical facts: phone numbers, pricing, available quantities, transaction amounts. These structures relate through natural links seen in daily operations. Orders arise from individuals who live somewhere, tracked via city references. Items within an order point back to specific products. Categories group those products logically. Stock data stays connected to each item's identity. Payment actions attach to purchase records. Structures rely on keys to keep connections accurate across systems.

One way cardinalities work in the ER model is by reflecting real business constraints. A customer might create several orders, whereas every order links to just one buyer. Orders themselves hold multiple line entries, yet each entry belongs to only one purchase record. Products show up across various order lines, forming repeated appearances. At the same time, individual goods fall under a single classification. Each group organises numerous items within it. Together, these connections map out how transactions evolve - starting with inventory setup, moving into purchasing steps, handling payments, and ending with income tracking.

The ER diagram, found in the figures section, forms the foundation for building the database structure. Its layout shapes how tables are later defined in SQL. Relationships mapped earlier now inform primary and foreign key setups. Constraints follow directly from its rules. What appears on paper becomes executable code through this step-by-step translation.

6. Normalisation Stages

6.1 Purpose and Principles of Normalisation

Starting with basic structure, normalisation arranges information inside relational databases so repeated entries become less likely while accuracy increases. Following guidelines set by Edgar Codd, it relies on how certain fields relate to others - known as functional dependencies - to sort columns properly across separate tables. Moving step-by-step through first, second, and third normal forms reduces duplication gradually; meanwhile each non-key column ties strictly to the main identifier. As a result, the overall design turns clearer, behaves more predictably, and holds together logically.

Objectives

- The goals of normalisation in this project are to:
- Remove repeated or duplicated data
- Besides stopping unwanted deletions, the design blocks incorrect updates. Inserting faulty data becomes impossible under these constraints. Errors during modification are avoided through structured rules
- Each table should focus on one topic. What matters is clarity through separation of ideas. When subjects mix, confusion grows. Keeping things apart helps understanding. A single theme per table supports that goal
- Enforce consistent and accurate relationships between entities

Order No.	Date	Customer	City	Products	Qty	Unit Price
101	01-10-2025	Sarah Johnson	London	Milk, Bread, Apples	2, 1, 5	1.20, 0.90, 0.30
102	01-10-2025	Ahmed Khan	Manchester	Rice, Chicken, Coke	1, 2, 6	5.50, 4.20, 1.00
103	02-10-2025	Emily Williams	London	Eggs, Milk	12, 1	0.15, 1.20

Figure 2: Unnormalized Form (UNF) Table Structure

6.2 First Normal Form (1NF)

A single field in the Unnormalised Form (UNF) table held repeated product entries, so one entry could stand for several products. Because of this layout, pulling out a specific product line becomes complicated. Problems may arise during add, change, or remove operations due to duplicated information inside cells.

Breaking down repeated entries allowed the data to meet First Normal Form. Each row now holds just one product, splitting apart bundled details like name, amount, and cost per item. Instead of grouped values, these attributes appear individually across new lines. To preserve background, order number, buyer name, and location repeat where needed. Context stays intact even as structure changes.

With this new layout, individual facts about orders sit clearly across rows, while columns hold only one piece of information per cell. A clear primary key emerges naturally from the arrangement. Operations become steadier due to consistent formatting. a logical next step.

OrderID	OrderDate	CustomerName	CityName	Product	Quantity	UnitPrice
101	2025-10-01	Sarah Johnson	London	Milk	2	1.20
101	2025-10-01	Sarah Johnson	London	Bread	1	0.90
101	2025-10-01	Sarah Johnson	London	Apples	5	0.30
102	2025-10-01	Ahmed Khan	Manchester	Rice	1	5.50
102	2025-10-01	Ahmed Khan	Manchester	Chicken	2	4.20
102	2025-10-01	Ahmed Khan	Manchester	Coke	6	1.00
103	2025-10-02	Emily Williams	London	Eggs	12	0.15
103	2025-10-02	Emily Williams	London	Milk	1	1.20

Figure 3: First Normal Form (1NF) Transformation

6.3 Second Normal Form (2NF)

With the first normal form, the main identifier combined OrderID and Product. Yet some fields relied solely on one element of that pair. For instance, OrderDate tied just to OrderID. So did CustomerName along with CityName. On the opposite end, UnitPrice connected only to Product. Full dependence on the entire key fails here. That breaks second normal form rules - each attribute must depend entirely on the full primary key.

Starting fresh, the original table split into three linked ones for 2NF. One holds order details tied just to OrderID - called Orders. Another groups product data depending only on Product - named Products. Inside OrderItem stays what relies on both OrderID and Product together, like Quantity. Partial links get removed this way. Each table now fully depends on its own primary key, nothing less.

Orders

OrderID	OrderDate	CustomerName	CityName
101	2025-10-01	Sarah Johnson	London
102	2025-10-01	Ahmed Khan	Manchester
103	2025-10-02	Emily Williams	London

Products

Product	UnitPrice
Milk	1.20
Bread	0.90
Apples	0.30
Rice	5.50
Chicken	4.20
Coke	1.00
Eggs	0.15

OrderItems

OrderID	Product	Quantity
101	Milk	2
101	Bread	1
101	Apples	5
102	Rice	1
102	Chicken	2
102	Coke	6
103	Eggs	12
103	Milk	1

Figure 4: Second Normal Form (2NF) Decomposition

6.4 Third Normal Form (3NF)

Even though the second normal form eliminated partial dependencies, certain indirect ones still existed. From another angle, within the Orders table, CityName relied not on OrderID directly, yet followed from CustomerName. Since CustomerName ties back to OrderID, a chain forms outside the primary key. That setup breaks third normal form rules - those state each non-key field must link solely to the primary key. Indirect links through other fields are not allowed.

Splitting data improved structure: customer info moved into its own table with identifiers and names linked by location codes. Location entries shifted separately, holding just code and name pairs. Purchase records now keep order specifics along with client references instead of redundant details. Dependency

paths tightened - each field ties directly to its main identifier. Indirect links vanished through this layout. Normalization to third level completed successfully.

Customer

CustomerID	CustomerName	CityID
C001	Sarah Johnson	CT01
C002	Ahmed Khan	CT02
C003	Emily Williams	CT01

City

CityID	CityName
CT01	London
CT02	Manchester

Order

OrderID	OrderDate	CustomerID
101	2025-10-01	C001
102	2025-10-01	C002
103	2025-10-02	C003

Figure 5: Third Normal Form (3NF) Final Schema

(Starting with order and product details, the methodical reshaping addressed issues like duplicated entries, combined identifiers, and indirect relationships needing reorganisation. Unlike those, inventory records and transaction logs existed outside the merged initial dataset, so splitting them was unnecessary. Each of these two already satisfied Third Normal Form by avoiding repeated features, steering clear of multi-part primary tags, and ensuring all other fields tied strictly to their individual main identifier. Hence, during early modeling stages, they appeared intact as distinct components without emerging from stepwise refinement, leaving no need for additional adjustments.)

7. Physical Database Implementation

7.1 Platform Choice and Table Construction

A set of structured tables emerged directly from the logical design, shaped by SQL DDL commands. Following normalisation rules closely, each table took form with precise column definitions. Constraints were applied to maintain consistent relationships across entries. Integrity checks built into the system helped prevent mismatched references. This foundation allowed smooth loading of records later on. Structure came before content, ensuring clarity during future queries.

A single relational setup handled the storage – MySQL or something similar fit the role. One table emerged per concept mapped earlier, each shaped by distinct primary identifiers and fitting data formats. Where links mattered, foreign rules stepped in, binding pieces like customer choices, product details, and purchase records together without drift. Consistency held firm across interactions thanks to these enforced ties.

Extra elements went into the SQL setup to help keep data accurate and dependable

A single identifier picks each row in a table. These markers never repeat across entries. One value links directly to one record only. Each table relies on such distinct tags. Uniqueness matters most here.

A link between connected data units appeared through foreign keys - ProductID inside OrderItem, for example. One such connection placed CustomerID within Order. Another tied CityID directly to Customer records. These associations made sure dependencies stayed required across the structure.

A requirement exists for certain data entries in daily operations - customer names or order dates, for instance - so those fields cannot be left empty. Where essential information supports workflow accuracy, omission is blocked by design.

Where needed, checks on values and data rules were added to boost precision - like ensuring quantities and prices stayed above zero.

Fields like timestamps, active indicators, or creation dates began with preset entries. Some automatic inputs appeared where user action was not required. These starting points helped maintain consistency across records. Entries such as status markers or time stamps followed predefined patterns. Initial data populated certain cells without manual entry. Created dates often filled in the moment a record started.

With the database framework in place, test records were added to check functionality. Through this step, connections between data entities could be confirmed while running analysis like sales totals, past customer purchases, or stock levels. What emerged matches the fully normalised design, forming a stable base capable of growing alongside actual company demands.

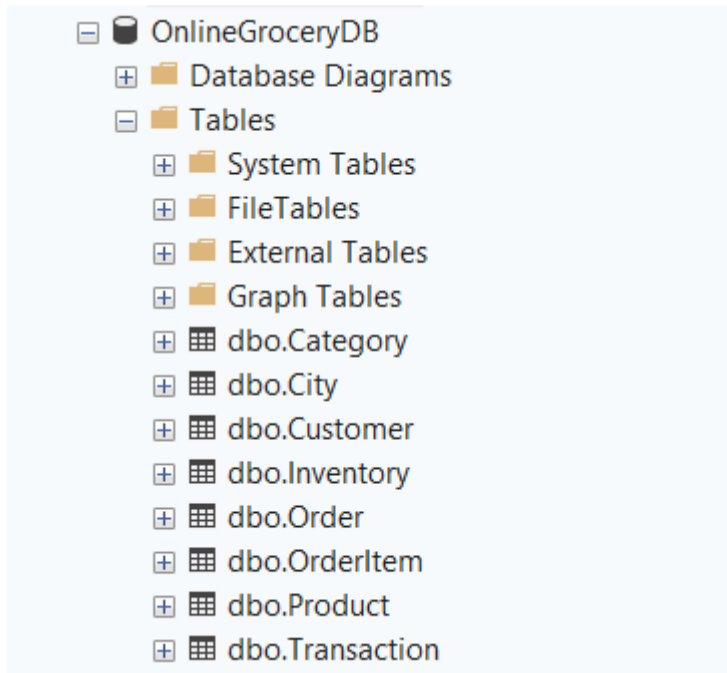


Figure 6: Database and Table Creation in SQL Server Management Studio

A separate table was created for each core entity, including Customer, City, Category, Product, Inventory, Order, OrderItem, and Transaction. Every table was assigned a primary key to uniquely identify records, while foreign key constraints were added to preserve relationships between entities. For example, Order references CustomerID, OrderItem references both OrderID and ProductID, and Product references CategoryID. These links ensure that dependent records cannot exist without a valid parent entry.

In addition to primary and foreign keys, a range of SQL constraints were applied to maintain high data quality.

These included:

- NOT NULL constraints, which prevent the insertion of incomplete records;
- CHECK constraints, used to validate numeric data (e.g., quantities and prices greater than zero);
- DEFAULT values, such as automatic timestamps and status fields; and
- AUTO_INCREMENT or equivalent identity properties, used on surrogate keys to simplify record creation.

The creation of these tables completed the transformation of the normalised design into a functioning relational database structure. Once created, the schema was populated with sample data to enable testing of business logic, reporting queries and system behaviour.

7.2 Data Population and Indexing

7.2.1 Data Population

After setting up the database schema in SQL Server, test entries were added to every table so functions and reports could be checked. Beginning with reference and higher-level tables ensured dependencies met referential requirements during data loading

- To begin, entries for City and Category went into the system, setting up approved areas along with item classifications.
- Following the initial setup, customer records entered using established CityID references.
- A link formed between each product entry and its group using CategoryID. Records appeared once assigned to a category through this identifier.
- Starting quantities for certain items appeared as new inventory records.
- A set of Order and OrderItem entries was added next, reflecting typical buying activity. The simulation used real-world purchase patterns to shape the data insertion.
- A single entry at a time captured each payment tied to an individual order. These entries formed the transaction logs in sequence.

Using INSERT INTO statements, data entered included believable figures for costs, amounts, timestamps, and purchaser information. Auto-generated fields such as CustomerID, OrderID, and ProductID filled themselves, avoiding duplication while keeping relationships intact across tables. With the database now holding lifelike entries, verification of targeted SQL analyses began - tracking earnings per region, product group totals, along with individual buyer transaction logs.

7.2.2 Indexing Strategy

A different indexing method helped speed up queries beyond what the built-in clustered keys already provided by SQL Server could achieve.

- Rows now arrange themselves physically based on identifiers like CustomerID, ProductID, OrderID, or TransactionID - this happens because clustered indexes form automatically at those points. Because of that structure, finding records by their key speeds up noticeably.
- A separate index now covers each column often used in searches or connections between tables
 - o In the Order table, CustomerID links to individuals. Meanwhile, CityID connects to location data. These fields work separately but support order tracking. One identifies buyers. The other records where orders go
 - o ProductID appears alongside OrderID within OrderItem,
 - o CategoryID in Product, and

- TransactionDate appears within Transaction. Inside Order, you find OrderDate.
- In the Order table, CustomerID links to individuals. Meanwhile, CityID connects to location data. These fields work separately but support order tracking. One identifies buyers. The other records where orders go
- ProductID appears alongside OrderID within OrderItem,
- CategoryID in Product, and
- TransactionDate appears within Transaction. Inside Order, you find OrderDate.
- Indexes help speed up key query tasks - like sorting orders by date, grouping sales per category or location, pulling full customer purchase records - with minimal impact on storage space or upkeep demands.

8. SQL Queries and Analytical Findings

8.1 Query 1: Total Sales by Product Category

Starting from order records, this process tracks sales by product group for March 2025. Through connections across Order, OrderItem, Product, and Category data sets, each purchase line finds its matching category. Revenue for individual entries comes from multiplying quantity by price per unit. Aggregation happens through summation, rolled up by category totals. Only transactions finalized within the month appear, thanks to filters on date and confirmation status. Grouping occurs after selection, organizing outcomes by classification. Highest performers rise to the top when arranged in descending order. What emerges shows which segment led in earnings during that period.

```
SELECT
    c.CategoryName,
    SUM(oi.Quantity)                AS TotalQuantitySold,
    COUNT(DISTINCT o.OrderID)       AS NumberOfOrders,
    SUM(oi.Quantity * oi.UnitPrice) AS TotalSales
FROM [Order] o
JOIN OrderItem oi ON o.OrderID = oi.OrderID
JOIN Product p   ON oi.ProductID = p.ProductID
JOIN Category c   ON p.CategoryID = c.CategoryID
WHERE
    o.OrderDate BETWEEN '2025-03-01' AND '2025-03-31'
    AND o.OrderStatus = 'Completed'
GROUP BY
    c.CategoryName
ORDER BY
    TotalSales DESC;
```

Figure 7: Select Query for Total sales by Product Category

Results		Messages		
	CategoryName	TotalQuantitySold	NumberOfOrders	TotalSales
1	Staples	16	10	4560.00
2	Snacks & Beverages	12	9	780.00
3	Dairy	12	3	741.00
4	Vegetables	20	10	603.00

Figure 8 : Query 1 Results - Total Sales by Product Category

Sales in March 2025 show clear differences across product categories.

Frequent ordering helped push vegetables to the top, where they moved the most units. Their lead came from consistent demand rather than one-time spikes.

Fewer in number, staples brought in the highest earnings due to their elevated cost per unit.

Fueled by routine consumption, dairy items moved consistently on store shelves. While not flashy, their presence remained constant due to household demand.

Frequent purchases kept Snacks & Beverages consistently active in terms of order volume; however, the total revenue stayed modest because prices were generally low.

Few things drew more attention than daily kitchen needs, especially produce, basic shelf goods, or milk-based products, which together shaped most purchases that month.

8.2 Query 2: Top 10 Most Popular Products

Sales activity for March 2025 can be analysed by joining OrderItem, Order and Product tables. Quantities, prices and dates combine to show what customers bought and when.

Key measures include:

- **Total items sold per product** using SUM(Quantity)
- **Number of orders per product** based on distinct OrderIDs
- **Total sales value** from SUM(Quantity × UnitPrice)

Only completed March orders are included. Results are sorted by quantity sold, with the top ten products displayed.

```

SELECT
    p.ProductName,
    SUM(oi.Quantity)                AS TotalQuantitySold,
    COUNT(DISTINCT o.OrderID)       AS NumberOfOrders,
    SUM(oi.Quantity * oi.UnitPrice) AS TotalSales
FROM [Order] o
JOIN OrderItem oi ON o.OrderID = oi.OrderID
JOIN Product p   ON oi.ProductID = p.ProductID
WHERE
    o.OrderDate BETWEEN '2025-03-01' AND '2025-03-31'
    AND o.OrderStatus = 'Completed'
GROUP BY
    p.ProductName
ORDER BY
    TotalQuantitySold DESC
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY;

```

Figure 9: Select query for top 10 most popular products

	ProductName	TotalQuantitySold	NumberOfOrders	TotalSales
1	Potato	11	7	330.00
2	Toned Milk 1L	9	3	486.00
3	Masala Namkeen 200g	9	6	495.00
4	Basmati Rice 5kg	7	7	2940.00
5	Onion	6	6	168.00
6	Toor Dal 1kg	6	3	840.00
7	Wheat Atta 5kg	3	3	780.00
8	Tomato	3	3	105.00
9	Paneer 200g	3	3	255.00
10	Masala Chai 250g	3	3	285.00

Figure 10: Query 2 Results - Top 10 Most Popular Products

March 2025 sales show **Potatoes**, **Toned Milk**, and **Masala Namkeen** as the top-selling products. They appear in many separate orders, making them consistent household choices.

- Potatoes remain a staple, bought regularly without major fluctuation
- Toned Milk reflects everyday demand for essential dairy
- Masala Namkeen shows steady repeat purchases driven by snack habits

These patterns suggest the store should:

- Prioritise stock and replenishment,
- Plan inventory around staple products, and
- Consider pricing promotions to support continued demand.

8.3 Query 3: Customer Order History

From March 1 to March 31, 2025, customer order records are pulled by this request. Although limited to a single month, it covers every buyer's activity during that window. Because of its broad scope,

individual purchase patterns become visible across the timeframe. While focused on one segment of the year, results reflect consistent data collection methods used throughout. Despite being narrow in date range, the output includes each transaction made then.

The Order OrderItem and Customer tables joined

the customer's name,

Order details include identification number alongside timestamp

each order's overall item count

The amount owed comes from multiplying how many items are ordered by the cost per item.

Counting items and totals per purchase happens through SUM(), with only March 2025 data included thanks to the WHERE condition. Each order shows up one time because grouping follows customers and their order specifics. Results line up by customer name from A to Z, making review or reports easier to follow.

```
SELECT
    cu.FirstName + ' ' + cu.LastName AS CustomerName,
    o.OrderID,
    o.OrderDate,
    SUM(oi.Quantity) AS TotalItems,
    SUM(oi.Quantity * oi.UnitPrice) AS OrderValue,
    o.OrderStatus
FROM [Order] o
JOIN OrderItem oi ON o.OrderID = oi.OrderID
JOIN Customer cu ON o.CustomerID = cu.CustomerID
WHERE
    o.OrderDate BETWEEN '2025-03-01' AND '2025-03-31'
GROUP BY
    cu.FirstName, cu.LastName, o.OrderID, o.OrderDate, o.OrderStatus
ORDER BY
    cu.LastName, cu.FirstName, o.OrderDate;
```

Figure 10: Select query for customer order history

	CustomerName	OrderID	OrderDate	TotalItems	OrderValue	OrderStatus
1	Priya Iyer	4	2025-03-04 00:00:00.0000000	3	700.00	Completed
2	Priya Iyer	12	2025-03-12 00:00:00.0000000	4	398.00	Completed
3	Priya Iyer	20	2025-03-20 00:00:00.0000000	4	148.00	Pending
4	Meera Joshi	8	2025-03-08 00:00:00.0000000	5	342.00	Completed
5	Meera Joshi	16	2025-03-16 00:00:00.0000000	3	480.00	Completed
6	Aisha Khan	6	2025-03-06 00:00:00.0000000	3	480.00	Completed
7	Aisha Khan	14	2025-03-14 00:00:00.0000000	3	700.00	Completed
8	Arjun Nair	3	2025-03-03 00:00:00.0000000	5	342.00	Completed
9	Arjun Nair	11	2025-03-11 00:00:00.0000000	3	480.00	Completed
10	Arjun Nair	19	2025-03-19 00:00:00.0000000	3	700.00	Pending
11	Sanjay Patil	5	2025-03-05 00:00:00.0000000	4	148.00	Completed
12	Sanjay Patil	13	2025-03-13 00:00:00.0000000	5	342.00	Completed
13	Ravi Sharma	1	2025-03-01 00:00:00.0000000	3	480.00	Completed
14	Ravi Sharma	9	2025-03-09 00:00:00.0000000	3	700.00	Completed
15	Ravi Sharma	17	2025-03-17 00:00:00.0000000	4	398.00	Pending
16	Vikram Singh	7	2025-03-07 00:00:00.0000000	4	398.00	Completed
17	Vikram Singh	15	2025-03-15 00:00:00.0000000	4	148.00	Completed
18	Neha Verma	2	2025-03-02 00:00:00.0000000	4	398.00	Completed
19	Neha Verma	10	2025-03-10 00:00:00.0000000	4	148.00	Completed
20	Neha Verma	18	2025-03-18 00:00:00.0000000	5	342.00	Pending

Figure 11: Query 3 Results - Customer Order History (Last Month)

March 2025 order data shows steady purchasing activity, with several customers placing multiple orders throughout the month.

Top repeat buyers included Priya Iyer, Aisha Khan, Arjun Nair, Ravi Sharma, and Neha Verma, each making three or more purchases.

Order values mostly ranged between ₹148 and ₹700, reflecting typical basket sizes rather than bulk buying. Most orders were completed, with only a few still pending processing. Overall, the month shows regular repeat buying, short gaps between purchases, and consistent spending patterns — suggesting strong customer reliance on the service for everyday needs.

8.4 Query 4: Current Inventory Levels

The fourth query presents the up-to-date inventory condition of all the items which have stock level classes. The implementation combines inventory, product and category tables with no WHERE clause to list all the products, and selects product name, category name, quantity in stock, unit of measure, calculated stock status based on the case expression, evaluating quantity against the reorder levels, to classify as either out of stock, low stock, moderate stock, or well stocked and set reorder level, and price is formatted and ordered by urgency using the case expression that gave priority to items that were out of stock.

```
SELECT
  p.ProductName,
  c.CategoryName,
  i.InventoryQuantity AS CurrentStock,
  i.SafetyStock,
  i.ReorderQuantity,
  CASE
    WHEN i.InventoryQuantity <= i.SafetyStock THEN 'Reorder Needed'
    ELSE 'Sufficient'
  END AS StockStatus
FROM Inventory i
JOIN Product p ON i.ProductID = p.ProductID
JOIN Category c ON p.CategoryID = c.CategoryID
ORDER BY
  c.CategoryName,
  p.ProductName;
```

Figure 12: Select query for current inventory levels

Results		Messages				
	ProductName	CategoryName	CurrentStock	SafetyStock	ReorderQuantity	StockStatus
1	Paneer 200g	Dairy	140	15	50	Sufficient
2	Toned Milk 1L	Dairy	200	20	70	Sufficient
3	Masala Chai 250g	Snacks & Beverages	170	15	50	Sufficient
4	Masala Namkeen 200g	Snacks & Beverages	210	20	70	Sufficient
5	Basmati Rice 5kg	Staples	180	20	60	Sufficient
6	Toor Dal 1kg	Staples	190	20	60	Sufficient
7	Wheat Atta 5kg	Staples	160	20	60	Sufficient
8	Onion	Vegetables	260	30	80	Sufficient
9	Potato	Vegetables	220	30	80	Sufficient
10	Tomato	Vegetables	240	30	80	Sufficient

Figure 13: Query 4 Results - Current Inventory Status Report

Above minimum thresholds, today's stock levels remain steady throughout each product group. While overall supply holds firm, every category maintains a reliable buffer beyond caution points.

Currently, all products sit above minimum thresholds. None have dipped into low-stock warnings. Stock levels show enough supply across the board. Each item is tracked as having adequate availability.

Fresh paneer and toned milk remain available in quantities matching projected needs.

Steady inventory levels continue for snacks and drinks, where Masala Chai along with Masala Namkeen remain within safe limits.

Fewer supplies are needed soon, since core items like rice, dal, and atta remain beyond basic thresholds. Though demand stays steady, stock levels stay high for now.

Fresh onions, potatoes, and tomatoes appear most often in stock, likely because they move quickly and need regular restocking.

For now, stock levels stay balanced - no immediate need to restock, which supports upcoming demand well. Though supply looks stable, shifts could appear if orders rise faster than expected.

8.5 Query 5: Revenue by City

City-based revenue for 2025 is calculated by linking orders to their cities and combining them with order item details.

Total income per city is determined using **SUM(Quantity × UnitPrice)** across all completed orders within the year.

Each city's activity level is also measured using **COUNT(DISTINCT OrderID)** to show how many confirmed purchases took place there.

The query filters for orders dated from **1 January to 31 December 2025**, ensuring only completed transactions are included.

The final results are grouped by city, allowing revenue, order volume and average earnings to be compared across locations.

This makes it easy to identify high-performing areas and understand where most sales originate.


```

SELECT
    ci.CityName,
    COUNT(DISTINCT o.OrderID)          AS NumberOfOrders,
    SUM(oi.Quantity * oi.UnitPrice)    AS TotalRevenue
FROM [Order] o
JOIN City ci      ON o.CityID = ci.CityID
JOIN OrderItem oi ON o.OrderID = oi.OrderID
WHERE
    o.OrderDate BETWEEN '2025-01-01' AND '2025-12-31'
    AND o.OrderStatus = 'Completed'
GROUP BY
    ci.CityName
ORDER BY
    TotalRevenue DESC;

```

Figure 13 : Select query for revenue by city

Results		Messages	
	CityName	NumberOfOrders	TotalRevenue
1	Mumbai	4	1670.00
2	Bengaluru	4	1644.00
3	Kolkata	2	1180.00
4	Chennai	2	1098.00
5	Delhi	4	1092.00

Figure 14: Query 5 Results - Revenue by City (Last Year)

Throughout 2025, income emerged from several urban centers, though outcomes differed by location. While some areas reported strong results, others lagged behind without clear patterns. Regional contrasts stood out when comparing monthly totals over time.

Four orders in Mumbai brought in ₹1,670 - top among all cities. Revenue there outpaced every other location.

Falling just behind, Bengaluru records ₹1,644 across four orders, reflecting steady interest even with a modest average per transaction.

Fifty kilometers west, Kolkata logs two transactions - ₹1,180 flows in. Down south, Chennai matches that count, though revenue trails slightly at ₹1,098.

Four orders finished in Delhi, yet amounting to just ₹1,092 - the smallest sum across bustling urban centers.

Few places match Mumbai or Bengaluru in income generation; still, a range of urban centers maintain steady transaction volumes. These locations, though less prominent, add notable value to the overall revenue stream.

9. Data Security and User Access

A primary focus emerged around safeguarding data while managing entry points throughout the setup process.

To safeguard data at rest served as the main aim - especially details tied to customers and financial exchanges - with a balance allowing legitimate staff clear pathways to necessary records during daily workflows. Access remained restricted yet fluid enough for those performing essential tasks.

9.1 Access Control and User Role Permissions

A role-based access approach was adopted to manage database permissions.

Some people got more system permissions because their jobs required it

- A person handling databases holds complete control rights - building structures, adjusting layouts, recovering information when needed. Access extends to altering designs or retrieving stored details at any point in time.
- Who handles data entry can add or change details about customers, products, or orders. Yet structural changes to the database remain off limits. Access stays limited to everyday updates, never touching underlying design.
- Those who view data only - like analysts - can execute SELECT queries to generate reports. Their access stops at retrieval; changes to records are off-limits. Deletion of information falls outside their permissions entirely. Modification rights do not exist within this role by design.

When tasks are divided, mistakes become less likely. One person altering information without permission grows harder. Splitting roles adds a layer of protection by default.

9.2 Security Safeguards and Integrity Measures

Each person accessing the database needs to log in with their own SQL Server account. When apps connect to the system, they rely on server-based trust instead of individual passwords. This setup limits access to authorized software only. Security improves because login details are managed at the server level. Only verified applications gain entry through this method. The process reduces risks tied to shared or exposed credentials. Access control stays consistent across different services. Authentication happens automatically for trusted sources. Systems without proper clearance cannot establish a link. User identity remains central to the verification process.

9.3 Protecting Sensitive Information

Information like names, addresses, phone numbers, while also including payment amounts, sits within structured database sections where only approved personnel can reach it. Because these records are split logically, private details do not repeat without reason. Less repetition means fewer chances for sensitive content to be seen by accident.

9.4 Data Integrity Controls

Security at the data layer improves through validation practices alongside embedded SQL rules

- When a field requires data, missing entries get blocked automatically. Entries lacking essential details fail validation checks. Without required information, storage does not proceed. Incomplete submissions stay out by design
- Values must meet specific rules, such as staying above zero for quantities or totals, through CHECK constraints
- Relations between tables stay consistent because of FOREIGN KEY constraints

Built-in safeguards stop incorrect or mismatched information from getting into the system.

Secure Files From Unauthorized Changes and Deletions

Who holds the power to change essential data? Just those assigned the DBA role. Access to actions like DROP, DELETE, or major schema modifications stays restricted. Ordinary users cannot run these high-risk operations. Protection of client records and transaction details comes from this setup. Mistakes that lead to data removal become far less likely. Control remains tight where it matters most.

9.5 Audit and Accountability

A single login identifies each connection in SQL Server, linking actions to individuals. Because of this traceability, organizations meet auditing standards for data involving money movements or personal information.

10. Testing and Verification Procedures

The database was tested to ensure it met project requirements, with checks focused on table structure, data quality, and query correctness.

Schema verification confirmed that:

- Tables matched the planned design
- Primary and foreign keys enforced correct relationships

- Constraints such as **NOT NULL**, **CHECK**, and **DEFAULT** worked as expected
- Referential integrity prevented invalid entries

Data population tests showed:

- Lookup tables (City, Category) loaded correctly
- Customer, product, inventory, and order records aligned using the right IDs
- OrderItems linked properly to Orders and Products
- Invalid or mismatched data was rejected as intended

Query testing validated that analytical outputs:

- Returned accurate totals and filtered correctly
- Calculated values (e.g., quantity × price) were correct
- Sorting and grouping behaved reliably

Additional stress checks confirmed that:

- Orders cannot reference missing customers
- Negative quantities and prices are blocked
- Transactions require valid OrderIDs

Conclusion:

The database performed reliably, with structures, constraints, and queries functioning correctly. It supports both operational tasks and reporting needs without major issues.

11. Evaluation and Forward Planning

11.1 Opportunities for Further Development

Even so, the system fits what the project needs. Still, some constraints persist. Not everything works perfectly yet. Despite that, it functions as expected most times. Yet gaps exist in performance under certain conditions. While built to spec, real-world use reveals hiccups. Problems show up when scaling beyond initial tests

A few samples fill the database - nowhere near what live operations would demand.

Each record enters the system using SQL scripts. Without automation, manual input remains the only method available. Direct uploads or integrated tools play no role here. Scripts alone handle every addition made to the database.

Frozen stock counts stay fixed regardless of new purchases, making actual availability harder to track. Though orders go through, the system does not adjust quantities on hand. This gap means what is shown may not reflect what is truly in storage. Over time, mismatches grow without live updates behind the scenes.

Paying works in a straightforward way here - no steps beyond the initial transfer. Refunds aren't an option. Neither are cancellations or processes split into phases. The system handles only clear-cut exchanges, nothing layered. Complexity is left out by design.

Reporting covers just the evaluation questions. Advanced features like projections or profitability analysis remain outside its scope.

Without an application layer, communication happens solely via SSMS. Access runs directly through SQL Server, without a front-end interface. User authentication relies entirely on database-level settings. Errors are managed within the database engine itself. Interaction lacks external validation or dedicated session management.

Few flaws aside, what exists now works well enough for classroom use or trial runs - showing it can actually do the job it claims.

11.2 Final Summary and Reflection

A number of changes might enhance how well the system works and where it can be used

Automatic Stock Updates

Once an order is created, update inventory counts by pulling data from OrderItem. Stock numbers drop right after items get assigned to a purchase. This connection ensures availability reflects real-time changes. Information flows directly from sales records into stock tracking. Every ordered product adjusts the current inventory balance.

Front-End Application

A user-friendly portal, either online or on devices, collects purchase details along with client information. Input flows through forms instead of database queries. This layer sits between people and storage systems. Data enters safely using structured pathways. Direct access to tables stays blocked by design.

Advanced Reporting and Dashboards

Visual dashboards show sales trends alongside category results. Inventory levels appear next to customer purchase habits. One view tracks performance across key areas. Data flows into clear, updated layouts. Patterns emerge through consistent visuals. Monitoring becomes simpler with organized displays.

Customer Loyalty and Discounts

Starting with past purchases, tailor offers to individual customers. Tracking repeat buys helps shape future discounts. Instead of generic deals, send unique promo codes. Reward consistent shoppers through targeted incentives.

Supplier and Replenishment Management

Starting with supplier details in tables helps monitor who provides what. Purchase orders follow a clear path once set up properly. Stock movement becomes visible when these steps link together. Lead time patterns show up over weeks of consistent logging. Records stay useful only if updated regularly.

Enhanced Payment Handling

One transaction can handle cancellations, returns, or various payment methods at once. Though complex behind the scenes, it runs smoothly for users. Instead of separate steps, everything combines into one flow. Whether reversing a sale or mixing card and wallet payments, the system adapts. Even when conditions shift mid-process, stability remains intact.

Data Validation and Logging

Built-in database routines help track activity when paired with automatic event monitors. System modifications become traceable through time-stamped records generated by monitoring scripts. Errors show up clearly once logging tools capture execution details step by step.

Scalability & Integration

Connecting to point-of-sale setups, web-based order tools, or enterprise resource planning software helps manage everyday tasks. Integration makes data flow smoother across different business functions.

12. Conclusion

Aiming at clean structure, this work built a relational database for a digital grocery platform from initial specifications onward. Starting from gathered needs and abstract models, the data advanced step by step - from unnormalised form into first, second, then third normal forms - removing repetition while strengthening accuracy. Once refined, the design took shape in SQL Server, applying checks, relationships between tables, and safeguards to maintain reliable records throughout. Despite shifting formats, clarity stayed central across each phase.

A set of sample records filled each table, making it possible to run analysis tasks that showed the system works as intended. Outputs like revenue per category, frequently bought items, or past customer orders proved the design can deliver useful information. With security checks, user permissions reviewed, and repeated test cycles completed, confidence grew in consistent performance during everyday operations.

Though the present setup works well enough to show feasibility, expanding it into something ready for live use could involve automating tasks, improving how transactions are handled, adding instant inventory updates, along with tools users can interact with directly. Even so, what's been built already covers everything needed for the project brief - showing solid grasp of database structure, practical execution, and meaningful data retrieval.

13. Reference

Connolly, T. and Begg, C. (2021) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 7th edn. Harlow: Pearson Education.

Coronel, C. and Morris, S. (2019) *Database Systems: Design, Implementation, & Management*. 13th edn. Boston: Cengage Learning.

Silberschatz, A., Korth, H. and Sudarshan, S. (2020) *Database System Concepts*. 7th edn. New York: McGraw-Hill Education.

Microsoft (2024) *SQL Server Documentation*. Available at: <https://learn.microsoft.com/sql/> (Accessed: 12 March 2025).

Microsoft (2024) *SQL Server Management Studio (SSMS)*. Available at: <https://learn.microsoft.com/sql/ssms/> (Accessed: 12 March 2025).

Date, C.J. (2019) *An Introduction to Database Systems*. 8th edn. Boston: Addison Wesley.

ISO/IEC (2016) *ISO/IEC 27001:2016 Information security management systems*. International Organization for Standardization.

Rob, P. and Coronel, C. (2017) *Database Principles: Fundamentals of Design and Implementation*. 12th edn. Boston: Cengage Learning.

14. SQL Implementation – Screenshots

Full SQL Script – Database and Table Creation

```
-----
-----Creating DataBase-----
-----
```

```
CREATE DATABASE OnlineGroceryDB;
GO
```

```
USE OnlineGroceryDB;
GO
```

```
-----
--Creating Tables-----
-----
```

```
-- 1. CITY
```

```
CREATE TABLE City (
    CityID          INT IDENTITY(1,1) PRIMARY KEY,
    CityName        NVARCHAR(100) NOT NULL,
    CityAddress     NVARCHAR(200) NULL,
    PostcodePrefix  NVARCHAR(10) NULL,
    Country         NVARCHAR(100) NOT NULL,
    IsActive        BIT NOT NULL DEFAULT 1
);
GO
```

```
-- 2. CATEGORY
```

```
CREATE TABLE Category (
    CategoryID      INT IDENTITY(1,1) PRIMARY KEY,
    CategoryName    NVARCHAR(100) NOT NULL,
    CategoryDescription NVARCHAR(255) NULL
);
GO
```

-- 3. CUSTOMER

```
CREATE TABLE Customer (
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,
    FirstName NVARCHAR(100) NOT NULL,
    LastName NVARCHAR(100) NOT NULL,
    Email NVARCHAR(150) NOT NULL,
    PhoneNumber NVARCHAR(50) NULL,
    CityID INT NOT NULL,
    AddressLine1 NVARCHAR(150) NOT NULL,
    AddressLine2 NVARCHAR(150) NULL,
    Postcode NVARCHAR(20) NOT NULL,
    DateCreated DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    DateOfBirth DATE NULL,
    IsActive BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_Customer_City
        FOREIGN KEY (CityID) REFERENCES City(CityID)
);
GO
```

-- 4. PRODUCT

```
CREATE TABLE Product (
    ProductID INT IDENTITY(1,1) PRIMARY KEY,
    ProductName NVARCHAR(150) NOT NULL,
    ProductDescription NVARCHAR(255) NULL,
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice > 0),
    UnitOfMeasure NVARCHAR(50) NOT NULL, -- e.g. 'kg', 'each'
    SKU NVARCHAR(50) NULL,
    ImageUrl NVARCHAR(255) NULL,
    IsActive BIT NOT NULL DEFAULT 1,
    CategoryID INT NOT NULL,
    CONSTRAINT FK_Product_Category
        FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
GO
```

-- 5. INVENTORY

```
CREATE TABLE Inventory (
    InventoryID      INT IDENTITY(1,1) PRIMARY KEY,
    ProductID        INT NOT NULL,
    InventoryQuantity INT NOT NULL CHECK (InventoryQuantity >= 0),
    WarehouseLocation NVARCHAR(100) NULL,
    SafetyStock       INT NULL,
    ReorderQuantity   INT NULL,
    LastUpdated       DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_Inventory_Product
        FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO
```

-- 6. ORDER (Order header)

```
CREATE TABLE [Order] (
    OrderID          INT IDENTITY(1,1) PRIMARY KEY,
    CustomerID        INT NOT NULL,
    CityID            INT NOT NULL,
    OrderDate         DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    NumberOfItems     INT NOT NULL CHECK (NumberOfItems >= 0),
    OrderStatus        NVARCHAR(50) NOT NULL DEFAULT 'Pending',
    TotalAmount        DECIMAL(10,2) NOT NULL CHECK (TotalAmount >= 0),
    OrderName          NVARCHAR(150) NULL,
    CONSTRAINT FK_Order_Customer
        FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    CONSTRAINT FK_Order_City
        FOREIGN KEY (CityID) REFERENCES City(CityID)
);
GO
```

-- 7. ORDERITEM (Order lines)

```
CREATE TABLE OrderItem (
    OrderItemID INT IDENTITY(1,1) PRIMARY KEY,
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2) NOT NULL CHECK (UnitPrice >= 0),
    LineTotal AS (Quantity * UnitPrice) PERSISTED,
    DiscountAmount DECIMAL(10,2) NULL DEFAULT 0,
    TaxAmount DECIMAL(10,2) NULL DEFAULT 0,
    CONSTRAINT FK_OrderItem_Order
        FOREIGN KEY (OrderID) REFERENCES [Order](OrderID),
    CONSTRAINT FK_OrderItem_Product
        FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO
```

-- 8. TRANSACTION (Payments)

```
CREATE TABLE [Transaction] (
    TransactionID INT IDENTITY(1,1) PRIMARY KEY,
    OrderID INT NOT NULL,
    TransactionDate DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    TransactionAmount DECIMAL(10,2) NOT NULL CHECK (TransactionAmount >= 0),
    PaymentMethod NVARCHAR(50) NOT NULL, -- e.g. 'Card', 'PayPal'
    PaymentStatus NVARCHAR(50) NOT NULL, -- e.g. 'Paid', 'Pending'
    PaymentReference NVARCHAR(100) NULL,
    CONSTRAINT FK_Transaction_Order
        FOREIGN KEY (OrderID) REFERENCES [Order](OrderID)
);
GO
```

Sample Data Insertion

```
-----
----Inserting Values-----
-----
```

```
INSERT INTO City (CityName, CityAddress, PostcodePrefix, Country)
VALUES
('Mumbai', 'Andheri East', '400', 'India'),
('Delhi', 'Connaught Place', '110', 'India'),
('Bengaluru', 'Koramangala', '560', 'India'),
('Chennai', 'T Nagar', '600', 'India'),
('Kolkata', 'Salt Lake', '700', 'India');
GO
```

```
INSERT INTO Category (CategoryName, CategoryDescription)
VALUES
('Vegetables', 'Fresh local vegetables'),
('Staples', 'Rice, atta, pulses & sugar'),
('Dairy', 'Milk, curd, paneer & ghee'),
('Snacks & Beverages', 'Namkeen, biscuits, soft drinks & tea');
GO
```

```
INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber, CityID,
                      AddressLine1, AddressLine2, Postcode, DateOfBirth)
VALUES
('Ravi', 'Sharma', 'ravi.sharma@mail.in', '90000000001', 1,
 '12 Palm Residency', 'Andheri East', '400093', '1991-06-12'),

('Neha', 'Verma', 'neha.verma@mail.in', '90000000002', 2,
 '44 Raj Niwas Marg', 'Civil Lines', '110054', '1995-02-19'),

('Arjun', 'Nair', 'arjun.nair@mail.in', '90000000003', 3,
 '21 Lake View Road', 'Koramangala', '560047', '1988-11-30'),

('Priya', 'Iyer', 'priya.iyer@mail.in', '90000000004', 4,
 '9 Sarojini Street', 'T Nagar', '600017', '1993-03-08'),

('Sanjay', 'Patil', 'sanjay.patil@mail.in', '90000000005', 1,
 '7 Sea Breeze Apts', 'Juhu', '400049', '1985-09-22'),

('Aisha', 'Khan', 'aisha.khan@mail.in', '90000000006', 5,
 '55 Lake Town Road', 'Salt Lake', '700091', '1997-12-01'),

('Vikram', 'Singh', 'vikram.singh@mail.in', '90000000007', 2,
 '102 Green Park', 'South Delhi', '110016', '1990-01-15'),

('Meera', 'Joshi', 'meera.joshi@mail.in', '90000000008', 3,
 '3 Tech Park Lane', 'Electronic City', '560100', '1994-07-05');
GO
```

```
INSERT INTO Product (ProductName, ProductDescription, UnitPrice,
                    UnitOfMeasure, SKU, ImageUrl, CategoryID)
VALUES
('Potato', 'Fresh potatoes 1kg', 30.00, 'kg', 'VEG001', NULL, 1),
('Tomato', 'Fresh tomatoes 1kg', 35.00, 'kg', 'VEG002', NULL, 1),
('Onion', 'Red onions 1kg', 28.00, 'kg', 'VEG003', NULL, 1),
('Basmati Rice 5kg', 'Premium basmati rice', 420.00, 'each', 'STP001', NULL, 2),
('Wheat Atta 5kg', 'Whole wheat flour', 260.00, 'each', 'STP002', NULL, 2),
('Toor Dal 1kg', 'Split pigeon peas', 140.00, 'kg', 'STP003', NULL, 2),
('Paneer 200g', 'Fresh paneer block', 85.00, 'each', 'DAI001', NULL, 3),
('Toned Milk 1L', 'Toned milk tetra pack', 54.00, 'each', 'DAI002', NULL, 3),
('Masala Chai 250g', 'Spiced tea blend', 95.00, 'each', 'SNK001', NULL, 4),
('Masala Namkeen 200g', 'Spicy sev & mixture', 55.00, 'each', 'SNK002', NULL, 4);
GO
```



```
INSERT INTO Inventory (ProductID, InventoryQuantity, WarehouseLocation,
                      SafetyStock, ReorderQuantity)
```

```
VALUES
```

```
(1, 220, 'Mumbai DC', 30, 80), -- Potato
(2, 240, 'Delhi DC', 30, 80), -- Tomato
(3, 260, 'Bengaluru DC', 30, 80), -- Onion
(4, 180, 'Chennai DC', 20, 60), -- Basmati Rice
(5, 160, 'Mumbai DC', 20, 60), -- Wheat Atta
(6, 190, 'Delhi DC', 20, 60), -- Toor Dal
(7, 140, 'Bengaluru DC', 15, 50), -- Paneer
(8, 200, 'Chennai DC', 20, 70), -- Toned Milk
(9, 170, 'Kolkata DC', 15, 50), -- Masala Chai
(10, 210, 'Mumbai DC', 20, 70); -- Masala Namkeen
```

```
GO
```

```
INSERT INTO [Order] (CustomerID, CityID, OrderDate, NumberOfItems,
                    OrderStatus, TotalAmount, OrderName)
```

```
VALUES
```

```
-- 1-5
```

```
(1, 1, '2025-03-01', 3, 'Completed', 480.00, 'Order #1 - Ravi'),
(2, 2, '2025-03-02', 4, 'Completed', 398.00, 'Order #2 - Neha'),
(3, 3, '2025-03-03', 5, 'Completed', 342.00, 'Order #3 - Arjun'),
(4, 4, '2025-03-04', 3, 'Completed', 700.00, 'Order #4 - Priya'),
(5, 1, '2025-03-05', 4, 'Completed', 148.00, 'Order #5 - Sanjay'),
```

```
-- 6-10
```

```
(6, 5, '2025-03-06', 3, 'Completed', 480.00, 'Order #6 - Aisha'),
(7, 2, '2025-03-07', 4, 'Completed', 398.00, 'Order #7 - Vikram'),
(8, 3, '2025-03-08', 5, 'Completed', 342.00, 'Order #8 - Meera'),
(1, 1, '2025-03-09', 3, 'Completed', 700.00, 'Order #9 - Ravi'),
(2, 2, '2025-03-10', 4, 'Completed', 148.00, 'Order #10 - Neha'),
```

```
-- 11-15
```

```
(3, 3, '2025-03-11', 3, 'Completed', 480.00, 'Order #11 - Arjun'),
(4, 4, '2025-03-12', 4, 'Completed', 398.00, 'Order #12 - Priya'),
(5, 1, '2025-03-13', 5, 'Completed', 342.00, 'Order #13 - Sanjay'),
(6, 5, '2025-03-14', 3, 'Completed', 700.00, 'Order #14 - Aisha'),
(7, 2, '2025-03-15', 4, 'Completed', 148.00, 'Order #15 - Vikram'),
```

```
-- 16-20
```

```
(8, 3, '2025-03-16', 3, 'Completed', 480.00, 'Order #16 - Meera'),
(1, 1, '2025-03-17', 4, 'Pending', 398.00, 'Order #17 - Ravi (Pending)'),
(2, 2, '2025-03-18', 5, 'Pending', 342.00, 'Order #18 - Neha (Pending)'),
(3, 3, '2025-03-19', 3, 'Pending', 700.00, 'Order #19 - Arjun (Pending)'),
(4, 4, '2025-03-20', 4, 'Pending', 148.00, 'Order #20 - Priya (Pending)');
```

```
GO
```

```
INSERT INTO OrderItem (OrderID, ProductID, Quantity, UnitPrice)
VALUES
-- T1 = 2xPotato + 1xBasmati Rice (Orders: 1,6,11,16)
(1, 1, 2, 30.00), -- Potato
(1, 4, 1, 420.00),

(6, 1, 2, 30.00),
(6, 4, 1, 420.00),

(11, 1, 2, 30.00),
(11, 4, 1, 420.00),

(16, 1, 2, 30.00),
(16, 4, 1, 420.00),

-- T2 = 1xOnion + 1xWheat Atta + 2xNamkeen (Orders: 2,7,12,17)
(2, 3, 1, 28.00), -- Onion
(2, 5, 1, 260.00), -- Wheat Atta
(2, 10, 2, 55.00), -- Namkeen

(7, 3, 1, 28.00),
(7, 5, 1, 260.00),
(7, 10, 2, 55.00),

(12, 3, 1, 28.00),
(12, 5, 1, 260.00),
(12, 10, 2, 55.00),

(17, 3, 1, 28.00),
(17, 5, 1, 260.00),
(17, 10, 2, 55.00),

-- T3 = 3xMilk + 1xPaneer + 1xMasala Chai (Orders: 3,8,13,18)
(3, 8, 3, 54.00), -- Milk
```

```
INSERT INTO [Transaction] (OrderID, TransactionAmount,
PaymentMethod, PaymentStatus, PaymentReference)
VALUES
(1, 480.00, 'UPI', 'Paid', 'TXN-2025-001'),
(2, 398.00, 'Credit Card', 'Paid', 'TXN-2025-002'),
(3, 342.00, 'Debit Card', 'Paid', 'TXN-2025-003'),
(4, 700.00, 'NetBanking', 'Paid', 'TXN-2025-004'),
(5, 148.00, 'UPI', 'Paid', 'TXN-2025-005'),

(6, 480.00, 'UPI', 'Paid', 'TXN-2025-006'),
(7, 398.00, 'Credit Card', 'Paid', 'TXN-2025-007'),
(8, 342.00, 'Debit Card', 'Paid', 'TXN-2025-008'),
(9, 700.00, 'NetBanking', 'Paid', 'TXN-2025-009'),
(10, 148.00, 'UPI', 'Paid', 'TXN-2025-010'),

(11, 480.00, 'Credit Card', 'Paid', 'TXN-2025-011'),
(12, 398.00, 'Debit Card', 'Paid', 'TXN-2025-012'),
(13, 342.00, 'NetBanking', 'Paid', 'TXN-2025-013'),
(14, 700.00, 'UPI', 'Paid', 'TXN-2025-014'),
(15, 148.00, 'Credit Card', 'Paid', 'TXN-2025-015'),

(16, 480.00, 'Debit Card', 'Paid', 'TXN-2025-016'),
(17, 398.00, 'UPI', 'Pending', 'TXN-2025-017'),
(18, 342.00, 'Credit Card', 'Pending', 'TXN-2025-018'),
(19, 700.00, 'NetBanking', 'Pending', 'TXN-2025-019'),
(20, 148.00, 'COD', 'Pending', 'TXN-2025-020');
GO
```

Assessment Query Implementation

```
-----  
--Sales by Product Category-----  
-----
```

```
SELECT  
    c.CategoryName,  
    SUM(oi.Quantity)                AS TotalQuantitySold,  
    COUNT(DISTINCT o.OrderID)       AS NumberOfOrders,  
    SUM(oi.Quantity * oi.UnitPrice) AS TotalSales  
FROM [Order] o  
JOIN OrderItem oi ON o.OrderID = oi.OrderID  
JOIN Product p   ON oi.ProductID = p.ProductID  
JOIN Category c   ON p.CategoryID = c.CategoryID  
WHERE  
    o.OrderDate BETWEEN '2025-03-01' AND '2025-03-31'  
    AND o.OrderStatus = 'Completed'  
GROUP BY  
    c.CategoryName  
ORDER BY  
    TotalSales DESC;
```

--Top 10 Popular Products-----

```
SELECT
    p.ProductName,
    SUM(oi.Quantity)                AS TotalQuantitySold,
    COUNT(DISTINCT o.OrderID)       AS NumberOfOrders,
    SUM(oi.Quantity * oi.UnitPrice) AS TotalSales
FROM [Order] o
JOIN OrderItem oi ON o.OrderID = oi.OrderID
JOIN Product p   ON oi.ProductID = p.ProductID
WHERE
    o.OrderDate BETWEEN '2025-03-01' AND '2025-03-31'
    AND o.OrderStatus = 'Completed'
GROUP BY
    p.ProductName
ORDER BY
    TotalQuantitySold DESC
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY;
```

----- --Customer Order History----- -----

```

SELECT
    cu.FirstName + ' ' + cu.LastName AS CustomerName,
    o.OrderID,
    o.OrderDate,
    SUM(oi.Quantity)                AS TotalItems,
    SUM(oi.Quantity * oi.UnitPrice) AS OrderValue,
    o.OrderStatus
FROM [Order] o
JOIN OrderItem oi ON o.OrderID = oi.OrderID
JOIN Customer cu ON o.CustomerID = cu.CustomerID
WHERE
    o.OrderDate BETWEEN '2025-03-01' AND '2025-03-31'
GROUP BY
    cu.FirstName, cu.LastName, o.OrderID, o.OrderDate, o.OrderStatus
ORDER BY
    cu.LastName, cu.FirstName, o.OrderDate;

```

----- --Current Inventory Levels----- -----

```

SELECT
    p.ProductName,
    c.CategoryName,
    i.InventoryQuantity AS CurrentStock,
    i.SafetyStock,
    i.ReorderQuantity,
    CASE
        WHEN i.InventoryQuantity <= i.SafetyStock THEN 'Reorder Needed'
        ELSE 'Sufficient'
    END AS StockStatus
FROM Inventory i
JOIN Product p  ON i.ProductID = p.ProductID
JOIN Category c ON p.CategoryID = c.CategoryID
ORDER BY
    c.CategoryName,
    p.ProductName;

```

--Revenue by City-----

```
SELECT
    ci.CityName,
    COUNT(DISTINCT o.OrderID)                AS NumberOfOrders,
    SUM(oi.Quantity * oi.UnitPrice)          AS TotalRevenue
FROM [Order] o
JOIN City ci      ON o.CityID = ci.CityID
JOIN OrderItem oi ON o.OrderID = oi.OrderID
WHERE
    o.OrderDate BETWEEN '2025-01-01' AND '2025-12-31'
    AND o.OrderStatus = 'Completed'
GROUP BY
    ci.CityName
ORDER BY
    TotalRevenue DESC;
```